Q. Discuss the use of this, static and final keywords with suitable examples.

Java provides special keywords like **this, static, and final** to control object behavior, memory usage, and inheritance.

1. this Keyword

Definition

this refers to the current object of a class. It is used to access instance variables and methods of the current object.

Syntax

```
this.variableName;
this.methodName();
this();   // constructor call
```

Uses

- Refers to current object variables
- Resolves ambiguity between instance and local variables
- Calls current class constructor (this())

Example

```
class Student {
   int id;

   Student(int id) {
      this.id = id;   // refers to current object
   }

   void display() {
      System.out.println(id);
   }
}

public class Main {
   public static void main(String[] args) {
      Student s = new Student(101);
      s.display();
   }
}
```

Output

101

2. static Keyword

Definition

static is used for class-level members that are shared among all objects of the class. It helps in memory management.

Syntax

```
static dataType variableName;
static returnType methodName() { }
```

Uses

- Static variable → shared among all objects
- Static method → called without object
- Static block → executes once

Example

```
class Counter {
   static int count = 0;

   Counter() {
      count++;
   }
}

public class Main {
   public static void main(String[] args) {
      new Counter();
      new Counter();
      new Counter();

      System.out.println(Counter.count);
   }
}
```

Output

3

---

3. final Keyword

Definition

final is used to restrict modification. It can be applied to variables, methods, and classes.

Syntax

final dataType variableName = value;

final void methodName() { }

final class ClassName { }

Uses

- Final variable → constant value
- Final method → cannot be overridden
- Final class → cannot be inherited

Example

```
class Demo {
   final int x = 10;

   void show() {
      System.out.println(x);
   }
}

public class Main {
   public static void main(String[] args) {
      Demo d = new Demo();
      d.show();
   }
}
```

Output

10

Key Points

- this refers to current object
- static is shared by all objects
- final makes variables constant and prevents inheritance
- Static variables save memory by sharing data
- Final ensures security and fixed behavior

Difference Between this, static, and final

this	static	final
Refers to current object	Belongs to class (shared)	Restricts modification
Used with instance variables & methods	Used with variables, methods, blocks	Used with variables, methods, classes
Cannot be used in static context	Can be accessed without object	Value cannot be changed once assigned
Resolves ambiguity (local vs instance)	Memory shared among all objects	Acts as constant
Used inside constructors/methods	Used for common data	Prevents overriding/inheritance

---

Conclusion

The keywords this, static, and final are essential in Java for controlling object behavior, memory usage, and program security. They improve code clarity, efficiency, and reliability.

Q. What are the special characteristics of constructors in Java? Explain different types of constructors with example.

Definition

A constructor in Java is a special method used to **initialize objects**. It is automatically called when an object is created.

Special Characteristics of Constructors

- Constructor name must be **same as class name**
- **No return type** (not even void)
- Automatically invoked during object creation
- Used to initialize instance variables
- Can be **overloaded**
- If not defined, Java provides a **default constructor**
- Can use this() for constructor chaining
- Cannot be inherited

Types of Constructors in Java

1. Default Constructor

Definition

A constructor with **no parameters**. If not defined, Java provides it automatically.

Syntax

```
class ClassName {
   ClassName() {
      // initialization
   }
}
```

Example

```
class Demo {
   Demo() {
      System.out.println("Default Constructor");
   }
}

public class Main {
   public static void main(String[] args) {
      Demo d = new Demo();
   }
}
```

Output

Default Constructor

---

2. Parameterized Constructor

Definition

A constructor that accepts parameters to initialize variables.

Syntax

```
class ClassName {
   ClassName(type param) {
      // initialization
   }
}
```

Example

```
class Student {
    String name;
    int id;

    Student(String name, int id) {
        this.name = name;
        this.id = id;
    }

    void display() {
        System.out.println(name + " " + id);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Sweta", 68);
        s.display();
    }
}
```

Output

```
Sweta 68
```

---

3. Copy Constructor (User-defined)

Definition

A constructor that copies values from another object of the same class.

Syntax

```
ClassName(ClassName obj) {
    // copy values
}
```

Example

```
class Student {
    String name;
    int id;

    Student(String name, int id) {
        this.name = name;
```

```
        this.id = id;
    }

    Student(Student s) {
        this.name = s.name;
        this.id = s.id;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Sweta", 68);
        Student s2 = new Student(s1);

        System.out.println(s2.name + " " + s2.id);
    }
}
```

Output

Sweta 68

---

4. Private Constructor

Definition

A constructor declared as private cannot be accessed outside the class.

Syntax

```
private ClassName() {
    // code
}
```

Example

```
class Demo {
    private Demo() {
        System.out.println("Private Constructor");
    }

    static void show() {
        System.out.println("Access through static method");
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        // Demo d = new Demo(); // Not allowed
        Demo.show();
    }
}
```

Output

Access through static method

Difference Between Constructors

Default	**Parameterized**	**Copy**	**Private**
No parameters	Takes parameters	Takes object	Not accessible outside
Auto provided	User defined	User defined	Used for control
Basic initialization	Flexible	Duplicate object	Restricts object creation

Conclusion

Constructors are essential for initializing objects in Java. Different types like default, parameterized, copy, and private constructors provide flexibility and control in object creation.

Q. Discuss the significance of StringTokenizer class and write a Java program to extract numbers from a comma-separated string using StringTokenizer and find their sum.

Definition

StringTokenizer is a class in Java (from java.util package) used to **break a string into tokens** based on delimiters. It helps in parsing strings efficiently.

Significance of StringTokenizer

- Used to **split strings into tokens**
- Useful in **parsing structured data** (CSV, logs, input data)
- Supports **multiple delimiters**
- Faster for simple tokenization
- Maintains **current position internally** while parsing
- Provides basic parsing (acts like a simple scanner/lexer)

Syntax

StringTokenizer st = new StringTokenizer(string, delimiter);

Common Methods

- hasMoreTokens() → checks if more tokens exist
- nextToken() → returns next token
- countTokens() → returns number of tokens

Program: Extract numbers and find sum

Example Program

```
import java.util.StringTokenizer;

public class SumTokenizer {
    public static void main(String[] args) {

        String str = "10,20,30,40,50";
        StringTokenizer st = new StringTokenizer(str, ",");

        int sum = 0;

        while (st.hasMoreTokens()) {
            int num = Integer.parseInt(st.nextToken());
            sum = sum + num;
        }

        System.out.println("Sum = " + sum);
    }
}
```

Output

Sum = 150

Explanation

- Input string is split using delimiter ,
- Each token is extracted using nextToken()

- Converted to integer using Integer.parseInt()
- Added to compute total sum

Key Points

- Does not use regular expressions
- Works sequentially (token by token)
- Legacy class (modern alternative: split() method)
- Efficient for simple parsing tasks

Conclusion

StringTokenizer is a simple and efficient class used for breaking strings into tokens. It is widely used for extracting and processing data from structured strings like comma-separated values.

Q. Explain the concept of method overloading using suitable example.

Method overloading in Java allows a class to have **multiple methods with the same name but different parameters**, enabling **compile-time polymorphism**.

Properties

- Methods must have the **same name**
- Parameters must differ in **number, type, or order**
- Cannot overload using **return type alone**
- Compiler selects method based on **arguments passed**

Syntax

```
class ClassName {
   returnType methodName(type a);
   returnType methodName(type a, type b);
}
```

Ways to Achieve Method Overloading

1. Changing Number of Parameters

Example:

```
class Product {

   int multiply(int a, int b) {
      return a * b;
   }

   int multiply(int a, int b, int c) {
      return a * b * c;
   }
}

public class Main {
   public static void main(String[] args) {
      Product p = new Product();

      System.out.println(p.multiply(2, 3));
      System.out.println(p.multiply(2, 3, 4));
   }
}
```

Output

```
6
24
```

---

2. Changing Data Types of Parameters

Example

```
class Product {

   int prod(int a, int b) {
      return a * b;
   }

   double prod(double a, double b) {
      return a * b;
   }
}

public class Main {
   public static void main(String[] args) {
      Product p = new Product();

      System.out.println(p.prod(2, 3));
      System.out.println(p.prod(2.5, 3.0));
   }
}
```

Output

6
7.5

---

3. Changing Order of Parameters

Example

```
class Student {

    void display(String name, int id) {
        System.out.println(name + " " + id);
    }

    void display(int id, String name) {
        System.out.println(id + " " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();

        s.display("Sweta", 1);
        s.display(2, "Amiya");
    }
}
```

Output

Sweta 1
2 Amiya

Advantages

- Improves **code readability**
- Increases **flexibility**
- Avoids using multiple method names
- Supports **compile-time polymorphism**

Conclusion

Method overloading allows multiple methods with the same name but different parameter lists. It helps in writing flexible and readable programs and is an important feature of compile-time polymorphism in Java.

Q. Explain about String class and discuss various methods in String Class with an example.

The String class in Java is used to **create and manipulate sequences of characters**. It is one of the most commonly used classes in Java and is present in the java.lang package.

Characteristics of String Class

- **Immutable** → once created, cannot be changed
- **Thread-safe** due to immutability
- Stored in **String constant pool**
- Provides many **built-in methods**
- Implements interfaces like **CharSequence, Comparable, Serializable**

Syntax (Creating Strings)

```
String s1 = "Hello";            // literal
String s2 = new String("Java");   // using new keyword
```

Common String Methods

Method	Description	Example
length()	Returns length of string	"Java".length() → 4
charAt()	Returns character at index	"Java".charAt(1) → a
toUpperCase()	Converts to uppercase	"java".toUpperCase()
toLowerCase()	Converts to lowercase	"JAVA".toLowerCase()
equals()	Compares strings	"a".equals("a") → true
equalsIgnoreCase()	Ignores case	"java".equalsIgnoreCase("JAVA")
concat()	Joins strings	"Hi".concat("Java")
substring()	Extracts part of string	"Hello".substring(1,4)
contains()	Checks substring	"Java".contains("av")
replace()	Replaces characters	"Java".replace('a','o')
trim()	Removes spaces	" hi ".trim()

Example Program

```
public class StringDemo {
   public static void main(String[] args) {

      String str = "  hello java  ";

      System.out.println("Length: " + str.length());
      System.out.println("CharAt(1): " + str.charAt(1));
      System.out.println("Upper: " + str.toUpperCase());
      System.out.println("Lower: " + str.toLowerCase());
      System.out.println("Trim: " + str.trim());
      System.out.println("Substring: " + str.substring(2, 7));
      System.out.println("Replace: " + str.replace('a', 'o'));
      System.out.println("Equals: " + str.equals("hello java"));
   }
}
```

Output

```
Length: 14
CharAt(1):
Upper:   HELLO JAVA
Lower:   hello java
Trim: hello java
Substring: hello
Replace:   hello jovo
Equals: false
```

Additional Points

- Strings are **immutable**, so methods return new objects
- == compares references, equals() compares values
- Stored in **heap memory (String pool)**
- Widely used in input/output and text processing

Conclusion

The String class is an essential part of Java programming. It provides powerful methods for handling text efficiently and is widely used in real-world applications.

Q. Explain constructor overloading in Java with a suitable example.

Constructor overloading in Java allows a class to have **more than one constructor with different parameter lists**. The appropriate constructor is selected at **compile time** based on the arguments passed.

Need for Constructor Overloading

- To initialize objects in **different ways**
- To provide **default and custom values**
- To improve **flexibility and code readability**

Syntax

```
class ClassName {
   ClassName() { }

   ClassName(type a) { }

   ClassName(type a, type b) { }
}
```

Example Program

```
class Student {
   int id;
   String name;

   // Default constructor
   Student() {
      id = 0;
      name = "Unknown";
   }

   // Constructor with one parameter
   Student(int i) {
      id = i;
      name = "Default";
   }

   // Constructor with two parameters
   Student(int i, String n) {
      id = i;
      name = n;
   }
```

```
    void display() {
        System.out.println(id + " " + name);
    }
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student(101);
        Student s3 = new Student(102, "Ravi");

        s1.display();
        s2.display();
        s3.display();
    }
}
```

Output

```
0 Unknown
101 Default
102 Ravi
```

Using this() in Constructor Overloading

- Used to call one constructor from another
- Must be the **first statement**

Ex:

```
Student() {
    this(0, "Default");
}
```

Rules

- Constructor name must be **same as class name**
- Must differ in **number, type, or order of parameters**
- Cannot have a **return type**
- this() must be the **first statement**

Key Points

- Provides **multiple ways to create objects**
- Improves **code readability and flexibility**
- Reduces **code duplication**
- Happens at **compile time**

Conclusion

Constructor overloading allows a class to have multiple constructors with different parameters, enabling flexible and efficient object initialization in Java.

Q. Compare String, StringBuffer and StringBuilder in Java with suitable examples.

1. String

Definition

String is a class in Java used to represent a sequence of characters. String objects are **immutable**, i.e., once created, their content cannot be changed. Any modification results in the creation of a **new object**.

Syntax

```
String variableName;
variableName = "value";

String variableName = "value";
String variableName = new String("value");
```

Example Program

```
public class Demo {
   public static void main(String[] args) {

      String s = "Java";
      s = s.concat(" Programming");

      System.out.println(s);
   }
}
```

Output

Java Programming

2. StringBuffer

Definition

StringBuffer is a class used to create **mutable strings**, meaning the content can be modified without creating new objects. It is **thread-safe (synchronized)** and suitable for multi-threaded environments.

Syntax

```
StringBuffer variableName;
variableName = new StringBuffer();

StringBuffer variableName = new StringBuffer("value");
```

Example Program

```
public class Demo {
   public static void main(String[] args) {

      StringBuffer sb = new StringBuffer("Java");
      sb.append(" Programming");

      System.out.println(sb);
   }
}
```

Output

Java Programming

---

3. StringBuilder

Definition

StringBuilder is similar to StringBuffer but is **not thread-safe**. It is faster and preferred in single-threaded applications where performance is important.

Syntax

```
StringBuilder variableName;
variableName = new StringBuilder();

StringBuilder variableName = new StringBuilder("value");
```

Example Program

```
public class Demo {
   public static void main(String[] args) {

      StringBuilder sb = new StringBuilder("Java");
      sb.append(" Programming");

      System.out.println(sb);
   }
}
```

Output

Java Programming

Difference Table

String	**StringBuffer**	**StringBuilder**
Immutable	Mutable	Mutable
Creates new object on modification	Modifies same object	Modifies same object
Thread-safe	Thread-safe	Not thread-safe
Slower	Slower than StringBuilder	Fastest
Uses String pool	Uses heap	Uses heap

Key Points

- String is **immutable → safe but slow**
- StringBuilder is **fastest → not thread-safe**
- StringBuffer is **safe → slightly slower**
- All three are used for **string handling**

Conclusion

String, StringBuffer, and StringBuilder differ mainly in **mutability and performance**. Choosing the right one improves efficiency and memory usage in Java programs.

Q. Write a Java program to check if a given string is a Pangram or not.

A **Pangram** is a sentence that contains **all 26 letters of the English alphabet at least once**.

Approach

- Convert the given string to **lowercase**
- Traverse from 'a' to 'z'
- For each character, check if it is present in the string
- If any character is missing → Not Pangram
- If all characters are present → Pangram

Logic

- Use toLowerCase() for case-insensitive comparison
- Use indexOf() to check presence of each character
- Use a boolean flag to track result

Program

```
public class PangramCheck {
   public static void main(String[] args) {

      String str = "The quick brown fox jumps over the lazy dog";
      str = str.toLowerCase();

      boolean isPangram = true;

      for (char ch = 'a'; ch <= 'z'; ch++) {
         if (str.indexOf(ch) == -1) {
            isPangram = false;
            break;
         }
      }

      if (isPangram)
         System.out.println("Pangram");
      else
         System.out.println("Not Pangram");
   }
}
```

Output

Pangram

Key Points

- Loop checks all **26 alphabets**
- Works for both **uppercase and lowercase**
- Efficient and simple logic

Conclusion

A string is a pangram if it contains all alphabets from 'a' to 'z'. This approach checks each letter efficiently using string methods.

Q. Write a Java program to check if a given string is an Anagram or not.

Two strings are called **anagrams** if they contain the **same characters with the same frequency**, but possibly in a different order.

Example:
listen and silent

Approach

- Convert both strings to **lowercase**
- Remove spaces (if any)
- Check if lengths are equal
- Convert strings to **character arrays**
- Sort both arrays
- Compare the arrays

Logic

- If sorted strings are equal → **Anagram**
- Otherwise → **Not Anagram**

Program

```
import java.util.Arrays;

public class AnagramCheck {
    public static void main(String[] args) {

        String s1 = "listen";
        String s2 = "silent";

        // Convert to lowercase
        s1 = s1.toLowerCase();
        s2 = s2.toLowerCase();

        // Remove spaces
        s1 = s1.replaceAll(" ", "");
        s2 = s2.replaceAll(" ", "");

        // Check length
        if (s1.length() != s2.length()) {
            System.out.println("Not Anagram");
            return;
        }

        // Convert to char arrays
        char[] arr1 = s1.toCharArray();
        char[] arr2 = s2.toCharArray();

        // Sort arrays
        Arrays.sort(arr1);
        Arrays.sort(arr2);

        // Compare
        if (Arrays.equals(arr1, arr2))
            System.out.println("Anagram");
        else
            System.out.println("Not Anagram");
    }
}
```

Output

Anagram

Key Points

- Sorting makes comparison easy

- Case-insensitive check
- Works even if spaces are present
- Uses Arrays.sort() and Arrays.equals()

Conclusion

Two strings are anagrams if their sorted character sequences are equal. This method is simple and efficient for checking anagrams.

Q. Write a Java program to display the details of a person. Personal details should be given in one method and the qualification details in another method.

Class:

A **class** is a blueprint or template used to create objects. It defines **variables (data)** and **methods (functions)**.

Syntax

```
class ClassName {
   // variables
   // methods
}
```

Object:

An **object** is an instance of a class. It represents a real-world entity and is used to access class members.

Syntax

```
ClassName obj = new ClassName();
```

Example :

```
Person p = new Person();
```

- Person → Class
- p → Object

Approach

- Create a class Person
- Define two methods:

    - personalDetails() → to display personal information
    - qualificationDetails() → to display qualification
- Call both methods from main()

Program

```
class Person {

   void personalDetails() {
      String name = "Ravi";
      int age = 21;

      System.out.println("Name: " + name);
      System.out.println("Age: " + age);
   }

   void qualificationDetails() {
      String degree = "B.Tech";
      String branch = "CSE";

      System.out.println("Degree: " + degree);
      System.out.println("Branch: " + branch);
   }
}

public class Main {
   public static void main(String[] args) {

      Person p = new Person();

      p.personalDetails();
      p.qualificationDetails();
   }
}
```

Output

```
Name: Ravi
Age: 21
Degree: B.Tech
Branch: CSE
```

Key Points

- Methods are used for **separation of functionality**

- Improves **readability and modularity**
- Object is required to call non-static methods

Conclusion

This program demonstrates how to organize data using multiple methods, making the code structured and easy to understand.

Q. Write a Java program to count number of vowels and consonants in the given text.

Vowels are **a, e, i, o, u**.
All other alphabets are **consonants**.

Approach

- Convert the string to **lowercase**
- Traverse each character of the string
- Check:
  - If character is vowel → increment vowel count
  - If character is alphabet but not vowel → increment consonant count

---

Program

```
public class VowelConsonant {
   public static void main(String[] args) {

      String str = "Java Programming";
      str = str.toLowerCase();

      int vowels = 0, consonants = 0;

      for (int i = 0; i < str.length(); i++) {
         char ch = str.charAt(i);

         if (ch >= 'a' && ch <= 'z') {
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
               vowels++;
            else
               consonants++;
         }
      }

      System.out.println("Vowels = " + vowels);
```

```
        System.out.println("Consonants = " + consonants);
    }
}
```

Output

Vowels = 5
Consonants = 11

Key Points

- Uses toLowerCase() for uniform comparison
- Ignores spaces and special characters
- Checks only alphabets

Conclusion

The program counts vowels and consonants by checking each character and classifying it based on conditions.